進階程式設計
Advanced Program Design

L6

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

讀取單精度浮點數的16進制內容

1. 使用聯合Union，讓浮點數i和無號整數j共用一個記憶體空間。
2. 使用指標

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

typedef union {
    float i;
    uint32_t j;
} union_num;
int main(int argc, char *argv[]) {
    union_num f;

    scanf("%f", &f.i);
    printf("f.j = 0x%X\n",f.j);
    return 0;
}
```

```
0.1
f.j = 0x3DCCCCCD

--------------------------------
Process exited after 2.122 seconds with return value 0
Press any key to continue . . .
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    float f;
    uint32_t *ptr = (void*)&f;

    scanf("%f", &f);
    printf("*ptr = 0x%X\n", *ptr);
    return 0;
}
```

```
0.1
*ptr = 0x3DCCCCCD

--------------------------------
Process exited after 1.797 seconds with return value 0
Press any key to continue . . .
```

指標編譯問題

- 試圖把一個類型的指標指向不同類型的變數，在某些平台上會有編譯錯誤，

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main() {
    float i;
    uint8_t *ptr = &i;

    while(scanf("%f", &i) != EOF) {
        for(uint16_t loop=0; loop<4; loop++) {
            printf("ptr[%hd] = 0x%X\n",loop,ptr[3-loop]);
        }
    }
    return 0;
}
```

```
ERROR!
/tmp/kczYO1f5OB/main.c:
In function 'main':
/tmp/kczYO1f5OB/main.c:7:20:
error:
initialization of 'uint8_t *' {aka 'unsigned char *'} from incompatible pointer type 'float *' [-Wincompatible-pointer-types]
7 |     uint8_t *ptr = &i;
|                    ^


=== Code Exited With Errors ===
```

2種解決方法

1. 使用void *
2. 強制轉換(uint8_t*)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main() {
    float i;
    uint8_t *ptr = (void*)&i;

    while(scanf("%f", &i) != EOF) {
        for(uint16_t loop=0; loop<4; loop++) {
            printf("ptr[%hd] = 0x%X\n",loop,ptr[3-loop]);
        }
    }
    return 0;
}
```

使用void*

```
0.1
ptr[0] = 0x3D
ptr[1] = 0xCC
ptr[2] = 0xCC
ptr[3] = 0xCD
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main() {
    float i;
    uint8_t *ptr = (uint8_t*)&i;

    while(scanf("%f", &i) != EOF) {
        for(uint16_t loop=0; loop<4; loop++) {
            printf("ptr[%hd] = 0x%X\n",loop,ptr[3-loop]);
        }
    }
    return 0;
}
```

使用強制轉換

```
0.1
ptr[0] = 0x3D
ptr[1] = 0xCC
ptr[2] = 0xCC
ptr[3] = 0xCD
```

浮點數的比較問題

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    float i = 0.1;

    if(i == 0.1) {
        printf("IF\n");
    } else if(i == 0.1f) {
        printf("ELSE IF\n");
    } else {
        printf("ELSE\n");
    }
    printf("============================\n");
    printf("sizeof(i) = %d\n",sizeof(i));
    printf("sizeof(0.1) = %d\n",sizeof(0.1));
    printf("sizeof(0.1f) = %d\n",sizeof(0.1f));
    return 0;
}
```

```
ELSE IF
============================
sizeof(i) = 4
sizeof(0.1) = 8
sizeof(0.1f) = 4
```

數字是預設用double去儲存
0.1f才是用float去儲存

比較兩個浮點數值是否相等

➢ 比較兩個程式碼

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    float i, j, p;

    printf("Please enter the two float numbers(i,j): ");
    scanf("%f %f", &i, &j);

    p = i + j;
    if(p == 0.3) {
        printf("IF\n");
    } else {
        printf("ELSE\n");
    }
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    float i, j, p;

    printf("Please enter the two float numbers(i,j): ");
    scanf("%f %f", &i, &j);

    p = i + j;
    if(p == 0.3f) {
        printf("IF\n");
    } else {
        printf("ELSE\n");
    }
    return 0;
}
```

```
Please enter the two float numbers(i,j): 0.1 0.2
ELSE

--------------------------------
Process exited after 4.159 seconds with return value 0
Press any key to continue . . .
```

```
Please enter the two float numbers(i,j): 0.1 0.2
IF

--------------------------------
Process exited after 29.24 seconds with return value 0
Press any key to continue . . .
```

比較兩個浮點數值是否相等

- 利用 == 去比較兩個倍精度浮點數可能會有潛在風險

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    double i, j, p;

    printf("Please enter the two double numbers(i,j): ");
    scanf("%lf %lf", &i, &j);

    p = i + j;
    if(p == 0.3) {
        printf("IF\n");
    } else {
        printf("ELSE\n");
    }
    return 0;
}
```

```
Please enter the two float numbers(i,j): 0.1 0.2
ELSE

--------------------------------
Process exited after 3.507 seconds with return value 0
Press any key to continue . . .
```

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    double i, j, p, k = 0.3;

    scanf("%lf %lf", &i, &j);
    p = i + j;
    printf("p = %.40lf\nk = %.40lf\n",p,k);
    return 0;
}
```

印出i+j和k值

```
0.1 0.2
p = 0.3000000000000000400000000000000000000000
k = 0.2999999999999999900000000000000000000000

--------------------------------
Process exited after 2.647 seconds with return value 0
Press any key to continue . . .
```

浮點數的比較

- 利用float.h標頭檔定義FLT_EPSILON和DBL_EPSILON，去判斷兩個浮點數值是否在EPSILON誤差
  - 單精度浮點數誤差：FLT_EPSILON(IEC 60559) = 0.00000011920928955078
  - 倍精度浮點數誤差：DBL_EPSILON(IEC 60559) = 0.00000000000000022204
- 實務上，採用自行設計的EPSILON會比較適當，如#define MY_EPSILON 1e-8
- 結合絕對值fabsf(用於單精度浮點數)或fabs(用於倍精度浮點數)，參考如下程式碼所述，則可認定兩個浮點數值是相同。

```
#include <stdio.h>
#include <stdlib.h>
#include <float.h>

int main(int argc, char *argv[]) {
     printf("FLT_EPSILON(IEC 60559) = %.20lf\n",FLT_EPSILON);
     printf("DBL_EPSILON(IEC 60559) = %.20lf\n",DBL_EPSILON);
     return 0;
}
```

```
FLT_EPSILON(IEC 60559) = 0.00000011920928955078
DBL_EPSILON(IEC 60559) = 0.00000000000000022204
```

浮點數的比較

- 計算兩個倍精度浮點數的差值是否足夠小於DBL_EPSILON？
- 實務上，根據使用狀況自行定義誤差值會比較適當

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#include <float.h>
int main(int argc, char *argv[]) {
    double i, j, p;

    printf("Please enter the two double numbers(i,j): ");
    scanf("%lf %lf", &i, &j);

    p = i + j;
    if(fabs(p-0.3)<=DBL_EPSILON) {
        printf("IF\n");
    } else {
        printf("ELSE\n");
    }
    return 0;
}
```

```
Please enter the two double numbers(i,j): 0.1 0.2
IF


--------------------------------
Process exited after 2.905 seconds with return value 0
Press any key to continue . . .
```

使用類函式巨集去比較浮點數差異

- math.h定義類函式巨集，可以在比較浮點數時避免例外的風險。

比較	類函式巨集指令
(x) > (y)	isgreater(x, y)
(x) ≥ (y)	isgreaterequal(x, y)
(x) < (y)	isless(x, y)
(x) ≤ (y)	islessequal(x, y)
((x) < (y)) || (x) > (y))	islessgreater(x, y)

The End